

Evolução do Diagrama de Classes – Principais Alterações Estruturais

Descrição das principais alterações estruturais realizadas no diagrama:

1. O modelo original, representado por um fluxo funcional do sistema, foi evoluído para um diagrama de classes UML completo, focado na estrutura interna do software e nos relacionamentos entre objetos.
2. Foram criadas classes centrais para representar os principais elementos do domínio, como **Estoque**, **Movimentacao**, **HistoricoMovimentacao**, **Usuario** e **ValidacaoOperacoes**, substituindo a representação baseada apenas em processos e funcionalidades.
3. A gestão de produtos foi reorganizada através da aplicação do padrão **Factory Method**, introduzindo a classe abstrata **ProdutoAviacao** e suas especializações (**PecaSimulador**, **SoftwareSimulador** e **AcessorioVoo**), além da classe **ProdutoFactory** responsável pela criação dos objetos.
4. O controle de acesso ao estoque passou a utilizar o padrão **Proxy**, com a criação da interface **IEstoqueService**, da classe **EstoqueServiceReal** e do **ProxyEstoque**, permitindo maior controle e segurança nas operações realizadas pelo usuário.
5. Foi incorporado o padrão **Observer** para o sistema de notificações, adicionando o **EstoqueSubject**, a interface **IObserver** e os observadores **NotifFuncionario** e **NotifAdministrador**, possibilitando a comunicação automática de alterações no estoque.
6. A coordenação entre os módulos de entrada e saída foi centralizada através do padrão **Mediator**, com a criação da interface **IMediator**, da classe **EstoqueMediator** e dos módulos **ModuloEntrada** e **ModuloSaida**, reduzindo o acoplamento entre componentes do sistema.
7. A geração de relatórios foi aprimorada com a adoção do padrão **Visitor**, permitindo que diferentes tipos de relatórios sejam criados sem alterar as classes dos produtos existentes.
8. O tratamento de erros foi formalizado através da classe de exceção **EstoqueInsuficiente**, substituindo a simples indicação de falha existente no modelo anterior e tornando o sistema mais robusto para lidar com inconsistências de estoque.
9. Foram definidos atributos, métodos e relacionamentos explícitos entre as classes, incluindo associações, dependências, heranças e implementações de interfaces, proporcionando uma modelagem orientada a objetos mais completa, reutilizável e alinhada às boas práticas de desenvolvimento de software.